



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS,
DHARAN-8**

**A MINOR PROJECT
PROPOSAL
ON
DECENTRALIZED PREPAID SMART POWER GRID**

**SUBMITTED BY
PRATIMA CHAUDHARY (PUR080BCT064)
RABIN KATTEL (PUR080BCT067)
SABIN BASNET (PUR080BCT077)
SARBESH TIMSINA (PUR080BCT082)**

**SUBMITTED TO
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PURWANCHAL CAMPUS
DHARAN, NEPAL**

**SUBMISSION DATE
2083 / 02 / 27**

TABLE OF CONTENTS

1	INTRODUCTION	1
1.1	Background	1
1.2	Problem Definition	1
1.3	Objectives	1
1.4	Scope of Project	2
2	LITERATURE REVIEW	3
2.1	Historical Evolution	3
2.2	Algorithms and Models	3
2.2.1	Prepaid Smart Contracts	3
2.2.2	Remaining-Time Estimation	3
2.2.3	Explainable Tamper Detection	3
2.3	Data Processing and Feature Engineering	4
2.4	Gaps and Challenge	4
3	METHODOLOGY	5
3.1	Overview	5
3.2	Proposed Block Diagram	5
3.3	System Design	6
3.4	Tools and Techniques	6
3.5	Data Sources	7
3.5.1	Primary Sources	7
3.5.2	Synthetic Data (Machine Learning Training)	7

3.6	Implementation	7
3.6.1	Hardware Layer Implementation	7
3.6.2	Blockchain Ledger Implementation	7
3.6.3	Backend Middleware and ML Integration	8
3.7	Testing and Evaluation	8
4	EXPECTED RESULTS	9
5	EXPECTED SCHEDULE	10
	REFERENCES	11

INTRODUCTION

1.1 Background

Electricity distribution systems require reliable measurement, timely payment processing, and rapid identification of unsafe or abnormal consumption. Conventional prepaid systems often depend on a central application to store balances and decide whether a consumer remains connected. This creates a single operational point of failure and makes it difficult to independently verify balance deductions and disconnection decisions.

This project proposes a local prototype of a decentralized prepaid smart power grid. An ESP32-based meter sends consumption and tamper telemetry to a FastAPI backend. The backend validates the meter, records operational history in SQLite, and uses a Solidity smart contract on a local Hardhat network to maintain and deduct prepaid energy credits. A React web dashboard presents the current balance, load, meter status, and system events. A lightweight machine-learning component estimates the remaining service time, while electrical current comparison provides an explainable tamper signal.

1.2 Problem Definition

Small communities and prototype deployments need an affordable way to demonstrate prepaid electricity enforcement without requiring a production utility platform. A purely centralized implementation can hide the source of balance changes, continue operating when its ledger is inconsistent, and provide limited visibility into meter tampering. The problem addressed by this project is therefore the design and validation of an end-to-end prototype that links physical telemetry, transparent prepaid accounting, safety decisions, and human-readable monitoring in one workflow.

1.3 Objectives

- To design an ESP32-based smart meter prototype that periodically reports load, energy, current, and tamper status.
- To implement a FastAPI service that validates telemetry, stores meter history, and

coordinates the grid-control workflow.

- To implement a Solidity prepaid ledger that records meter credits and deducts energy consumption transparently.
- To provide automatic disconnect decisions when tampering is detected or prepaid credit is depleted.
- To estimate remaining operating time from historical consumption and display system status through a React dashboard.

1.4 Scope of Project

The project is limited to a working local prototype and simulation environment. It includes ESP32/Wokwi telemetry generation, meter registration and validation, SQLite history, local Hardhat blockchain transactions, prepaid balance deduction, current-differential tamper detection, remaining-hours estimation, and a React web dashboard. The default demonstration account represents a meter in Dharan.

The project does not claim to replace certified utility meters, production blockchain infrastructure, tariff regulation, utility-scale grid protection, or a production security audit. The local blockchain and seeded data are used to evaluate the proposed workflow in a controlled environment.

LITERATURE REVIEW

2.1 Historical Evolution

Utility metering has progressed from manually read electromechanical meters to digital meters and Advanced Metering Infrastructure (AMI), which supports frequent telemetry and remote monitoring. AMI improves collection and visibility but generally retains a centrally managed database and decision process. IoT devices, smart contracts, and data-driven monitoring provide complementary mechanisms for building more auditable and responsive prototypes. [1, 2]

2.2 Algorithms and Models

2.2.1 Prepaid Smart Contracts

A smart contract can represent prepaid credit as an auditable ledger and expose controlled functions for funding, deduction, and balance checks. In this project, the local Solidity contract is the source of truth for credit deductions, while the backend mirrors operational status in SQLite.

2.2.2 Remaining-Time Estimation

Regression models are commonly used to relate consumption history to future load. The proposed prototype uses a linear regression model trained on seeded Dharan meter history and combines its learned baseline with the current account balance to estimate remaining service time. The estimate is advisory; the contract balance remains authoritative. [3]

2.2.3 Explainable Tamper Detection

For a small prototype, a direct comparison between line and neutral current is easier to inspect and test than an opaque classifier. The system flags a reading when the absolute current differential exceeds a configurable threshold, and records the result with the meter event. More advanced anomaly models can be evaluated in future work. [4]

2.3 Data Processing and Feature Engineering

The system processes two distinct data pipelines: live analog telemetry from the ESP32 edge node and structured synthetic datasets used for training the machine learning models.

Engineered Feature	Data Type	Description and Purpose
Load _{curr}	Float	The real-time simulated power demand scaled to Kilowatts (kW).
Energy _{cum}	Float	The cumulative energy consumed over the session in Watt-hours (Wh).
Delta _{current}	Float	The absolute difference between line and neutral current, used as an explainable tamper indicator.
Hour _{tod}	Integer	A Time-of-Day index (0-23) assigned to the data point to capture cyclical daily consumption habits.
Bal _{token}	Float	The remaining prepaid balance read from the smart contract and used with the estimated consumption rate.

2.4 Gaps and Challenge

Existing AMI frameworks commonly rely on centralized operational services, while blockchain-only demonstrations do not control a physical load. This project addresses the prototype-level gap by connecting an ESP32 relay decision to validated telemetry and a prepaid blockchain transaction. The resulting workflow can be measured locally before larger-scale reliability, privacy, and security questions are considered.[4]

METHODOLOGY

3.1 Overview

The project follows an iterative prototype-development methodology. Meter telemetry is generated by an ESP32/Wokwi node, validated and processed by a FastAPI backend, recorded in SQLite, and connected to a local Hardhat smart contract for prepaid deduction. The backend returns a control command to the firmware, and the React dashboard reads the resulting system state. Historical seed data is used to initialize and evaluate the remaining-time estimator.

3.2 Proposed Block Diagram

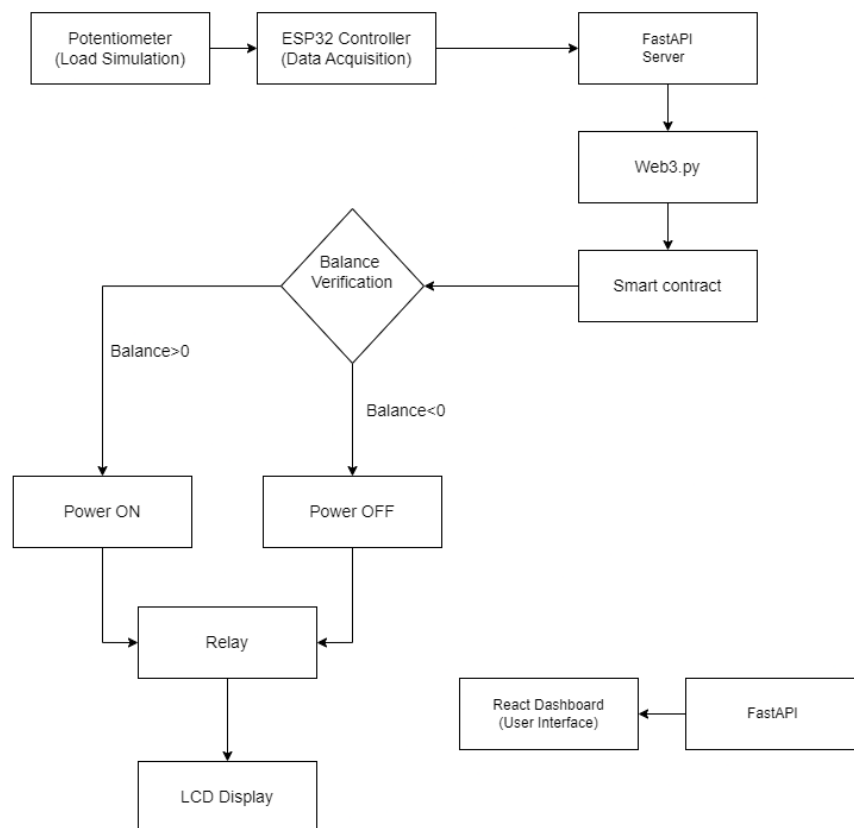


Figure-1: Proposed Block Diagram of decentralized power grid.[4]

3.3 System Design

The architecture is organized into four cooperating layers: the firmware layer for sensing and relay control, the backend layer for validation and orchestration, the blockchain layer for prepaid credit, and the dashboard layer for monitoring. SQLite stores local operational records without replacing the blockchain as the credit ledger.

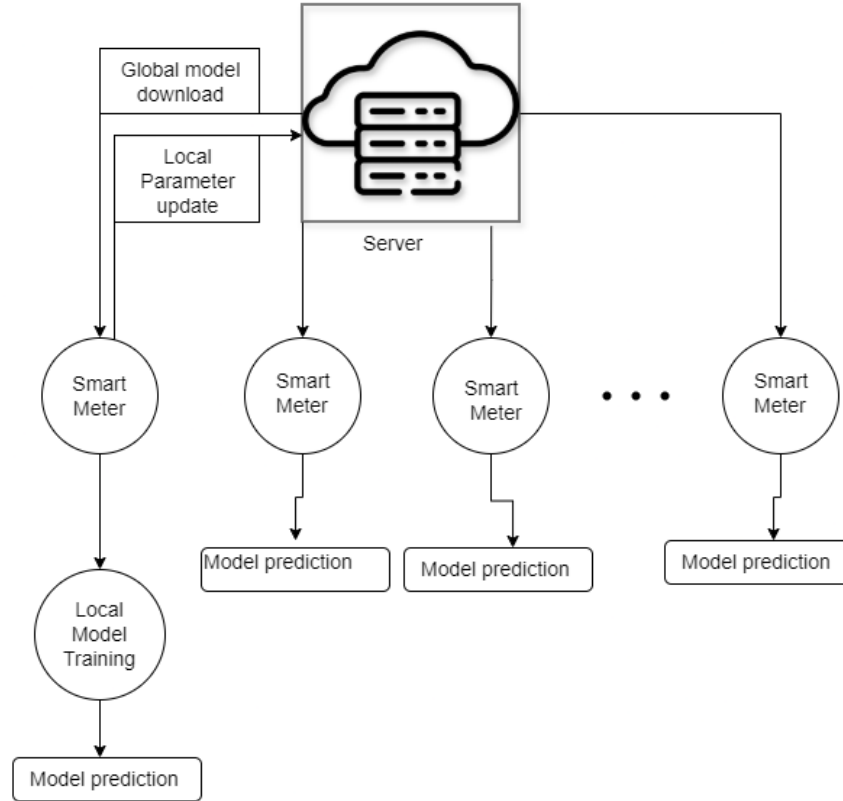


Figure-2: System Design for decentralized power grid. [5]

3.4 Tools and Techniques

- Hardware and simulation: ESP32, current/load sensors, relay, and Wokwi
- Software: PlatformIO/C++, FastAPI and Web3.py (Python), Hardhat and Solidity, React/Vite, SQLite, and scikit-learn

3.5 Data Sources

3.5.1 Primary Sources

Real-time prototype telemetry will be generated by the ESP32/Wokwi edge node. Test scenarios will include normal load, depleted credit, and tamper conditions produced through controlled sensor inputs.

3.5.2 Synthetic Data (Machine Learning Training)

The regression estimator will use the seeded Dharan meter history stored in CSV format. The dataset contains timestamped consumption and remaining-token values, from which time-of-day and day-of-week features are derived. Tamper detection will be evaluated using controlled line/neutral current differences and explicit tamper flags.

3.6 Implementation

3.6.1 Hardware Layer Implementation

The physical edge node is built around an ESP32 configured with PlatformIO. The firmware samples the meter inputs, constructs a JSON telemetry packet, sends it at a fixed interval, and uses the backend response to keep the relay active or isolate the circuit.

3.6.2 Blockchain Ledger Implementation

The transaction layer relies on a Solidity smart contract compiled and tested with Hardhat. The contract maps meter wallets to prepaid credits and exposes balance, funding, deduction, and zero-balance checks. For each accepted reading, the prototype applies a bounded integer deduction based on reported energy and returns the updated balance and isolation decision. Conceptually, the deduction is: [6]

$$\text{Bal}_{\text{new}} = \text{Bal}_{\text{current}} - \text{Cost}(\text{energy})$$

3.6.3 Backend Middleware and ML Integration

The coordination hub is built with FastAPI. It verifies the meter against SQLite, handles tamper and balance conditions, calls the contract through Web3.py, stores meter history, and returns a relay command. The ML module fits a linear regression baseline for service-time estimation, while current-differential classification provides the tamper flag.[7]

3.7 Testing and Evaluation

Testing will be performed at unit, integration, and demonstration levels. Unit tests will verify balance arithmetic, meter validation, current-differential classification, and remaining-time calculations. Integration tests will exercise the telemetry-to-contract-to-relay response flow using a local Hardhat node. The prototype will be evaluated using transaction response time, balance deduction correctness, remaining-time estimation error, tamper-detection precision, and correct isolation behavior for depleted credit.

EXPECTED RESULTS

The final prototype is expected to demonstrate an integrated prepaid grid loop. The ESP32/Wokwi node will send JSON telemetry to the FastAPI gateway, which will validate the meter, process the reading, deduct prepaid credit through the local smart contract, and return a relay command. Tampering or depleted credit will produce an isolation decision that the firmware can apply to the simulated load circuit.

The React web dashboard will display current users, load, meter status, balance, telemetry history, tamper alerts, and an estimated number of hours remaining. The estimate will be presented as a forecast rather than an exact guarantee, because actual exhaustion depends on future consumption. Controlled current differences and explicit tamper inputs will generate explainable anomaly alerts.

The framework's final performance will be quantitatively evaluated using four core engineering metrics:

- Smart Contract Transaction Latency: Time between an accepted telemetry update and local ledger deduction confirmation.
- Balance Deduction Accuracy: Mathematical precision of contract-executed tariff calculations versus theoretical values.
- Remaining-Time Forecast Error: The accuracy of the service-time estimate, evaluated using Root Mean Squared Error (RMSE) where labelled test data is available.
- Tamper Detection Precision: The success rate of the current-differential rule in identifying controlled tamper cases with minimal false alarms.

EXPECTED SCHEDULE



Figure-3: Gantt Chart

REFERENCES

- [1] Smith, J., & Kumar, A. (2022). IoT-Based Smart Energy Meter for Real-Time Monitoring and Automated Billing. *International Journal of Smart Grid Technologies*, 10(3), 150-158.
- [2] Zhao, L., & Chen, Y. (2020). Real-Time Energy Consumption Monitoring Using IoT and Cloud Computing. *IEEE Transactions on Industrial Informatics*, 16(7), 4562-4570.
- [3] Mulani, A. O., & Mane, P. B. (2017). Watermarking and cryptography based image authentication on reconfigurable platform. *Bulletin of Electrical Engineering and Informatics*, 6(2), 181-187
- [4] J. Jithish, B. Alangot, N. Mahalingam, and K. S. Yeo, "Distributed Anomaly Detection in Smart Grids: A Federated Learning-Based Approach," *IEEE Access*, vol. 11, pp. 9824–9838, 2023.
- [5] Choudhary, N., & Singh, R. (2020). Implementation of IoT-Based Prepaid Smart Energy Meter. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 11(5), 310-316
- [6] Gholizadeh, N., Musilek, P., 2022. Federated learning with hyperparameter-based clustering for electrical load forecasting. *Internet of Things* 17, 100470. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2542660521001104>, doi:10.1016/j.iot.2021.100470.
- [7] Husnoo, M.A., Anwar, A., Hosseinzadeh, N., Doss, R., Sikdar, B., 2023b. Misbehaviour detection for smart grids using a privacy-centric and computationally efficient federated learning approach, in: *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, pp. 2269–2274.